# Artificial and Computational Intelligence

Assignment 1 - Theory Document

Problem Statement 0-B (Travelling Agent)

Student Name: Gadamsetty V Sai Manoj Kumar

BITS ID: 2025MT12100

Date: 11 April 2026

## 1. Agent Environment and PEAS

### Problem Context

A travelling agent must move from Chennai (node A) to Hyderabad (node X) on a weighted graph. Edge weights represent approximate travel time between cities. The task is to identify a valid route to the goal using the required search methods (IDA* and Hill Climbing), while reporting path and complexity.

### PEAS Description

Performance Measure:Reach goal node X with minimum travel cost; produce valid path; report time and space complexity.

Environment:Weighted graph of cities, static during one run, fully observable.

Actuators:Move from current node to an adjacent node connected by an edge.

Sensors:Current node, neighboring nodes, and edge costs from graph representation.

### Environment Characteristics

- Fully observable: all nodes/edges/costs are known from the start.
- Deterministic: moving on an edge always gives the same next node and cost.
- Sequential: each decision affects subsequent choices in the path.
- Static: topology and weights do not change during execution.
- Discrete: nodes and transitions are finite and countable.

## 2. Heuristic / Fitness Function Definition

For this problem, the heuristic follows the assignment statement exactly:

h(n) = average of total travel costs across all simple paths from node n to goal node X.

Interpretation:

- Lower h(n) indicates that node n is generally closer (in travel-time sense) to X.
- The heuristic is computed from graph structure, not manually guessed.

Usage in algorithms:

- IDA*: uses f(n) = g(n) + h(n), where g(n) is path cost from start to n.
- Hill Climbing: selects neighbor with best (lowest) heuristic value; if no improving neighbor exists, local optimum behavior is reported.

## 3. Data Structures Used in Implementation

- Graph: Dictionary of dictionaries (adjacency map with edge weights).

Example: graph[A][D] = 3

- Path containers: list for ordered path sequence.
- Visited tracking: set to prevent node revisits in simple-path exploration.
- Heuristic table: dictionary mapping node -> heuristic value h(n).
- Result object: structured record containing path, cost, expanded nodes, depth, and flags.

## 4. Informed and Local Search Methods (Summary)

### IDA* (Informed Search)

- Starts with threshold = h(start).

- Performs depth-first bounded search using f(n).
- If goal not found, threshold increases to minimum exceeded f-cost and repeats.
- Returns path A -> D -> G -> X (cost 19) in the submitted implementation run.

Hill Climbing (Local Search)

- At each step chooses best heuristic neighbor.
- Tracks local optimum condition when strict heuristic improvement is not possible.
- Submitted run reports path A -> D -> G -> X and marks local-optimum encountered as Yes.

5. Complexity Reporting Approach

The code prints complexity information explicitly:

- IDA*: Time O(b^d) worst case, Space O(bd) worst case.
- Hill Climbing: Time O(b) per move, Space O(b) for neighbor inspection.

6. Submission Note

This document contains theory only (PEAS, environment analysis, heuristic definition, and method explanation).

Implementation is provided separately in Python notebook / code as required.